

108 AI Platform — Manuale Pratico di Installazione

Un'AI che ti conosce. Knowledge a grafo e vettoriale. Agenti personalizzati. Fino all'80% di risparmio rispetto al mercato.

Indice

- Cosa stai installando
- Prerequisiti
- Installazione locale (dev/test)
- Installazione su server cloud (produzione)
- Architettura: perché costa meno e performa meglio
- Controllo costi: strategie intelligenti e predittive
- Come funziona nel concreto: ti guido
- Configurazione Provider AI (DeepSeek + Alibaba)
- Troubleshooting

1. Cosa stai installando

108 AI Platform è un sistema AI privato multi-tenant che offre:

Capacità | Come funziona

AI che ti conosce | Knowledge base vettoriale (Qdrant) + grafo relazionale (Neo4j) = l'AI ricorda chi sei, cosa fai, come lavori

Agenti personalizzati | Template configurabili per ruolo: email, documenti, customer support, vendite, HR, legale, marketing

Costi 30-50% del mercato | Routing intelligente su 6 tier di modelli (DeepSeek + Qwen) invece di un unico modello costoso

Controllo totale | I tuoi dati restano sul tuo server. Nessun vendor lock-in. Export sempre disponibile

Stack tecnologico



2. Prerequisiti

Per installazione locale (Windows/Mac/Linux)

Requisito | Minimo | Consigliato

- Docker Desktop | v24+ | v27+
- Docker Compose | v2.20+ | v2.29+
- Node.js | 20 LTS | 22 LTS
- npm | 10+ | 10+
- RAM | 8 GB | 16 GB
- Disco | 10 GB liberi | 20 GB
- OS | Windows 10/11, macOS 12+, Ubuntu 22+ | -

Per installazione cloud (produzione)

Requisito | Specifica

- VPS | Hetzner CX31 o superiore (4 vCPU, 8 GB RAM, 80 GB SSD)
- OS | Ubuntu 24.04 LTS (clean)
- Dominio | Un dominio con accesso DNS (es. tuodominio.it)
- API Key DeepSeek | Gratuita: <https://platform.deepseek.com/api-keys>
- API Key Alibaba (DashScope) | Gratuita: <https://dashscope.console.aliyun.com/>

Costo infrastruttura cloud

Voce | Costo/mese

- Hetzner CX31 (4 vCPU, 8GB) | ~8 EUR
- Backup automatici Hetzner | ~2 EUR
- Dominio (.it/.dev) | ~1 EUR (ammortizzato)
- Totale infra | ~11 EUR/mese

Il costo LLM è separato e dipende dall'uso (vedi sezione 6).

3. Installazione locale

3.1 Clone e setup iniziale

```
# Clona il repository
git clone <REPO_URL> aia-platform
cd aia-platform

# Setup automatico (crea .env, installa dipendenze)
make setup
```

3.2 Configura le API key

Apri il file .env appena creato e inserisci le chiavi:

```
# .env — MINIMO per funzionare in locale:

# Database
POSTGRES_USER=aia
POSTGRES_PASSWORD=una_password_sicura_qui
POSTGRES_DB=aia_platform
DATABASE_URL=postgresql://aia:una_password_sicura_qui@postgres:5432/aia_platform

# Redis
REDIS_URL=redis://redis:6379

# Qdrant
QDRANT_URL=http://qdrant:6333

# LiteLLM (chiave admin locale — cambiala in produzione)
LITELLM_MASTER_KEY=sk-108ai-dev-local
LITELLM_URL=http://litellm:4000

# === API KEY AI — OBBLIGATORIE (almeno una) ===
DEEPSEEK_API_KEY=sk-la-tua-chiave-deepseek
DASHSCOPE_API_KEY=sk-la-tua-chiave-alibaba

# Opzionali (per embedding fallback)
OPENAI_API_KEY=sk-xxx

# App
NODE_ENV=development
JWT_SECRET=un_segreto_lungo_almeno_32_caratteri_qui
PORT=3000
```

3.3 Avvia tutto

```
# Avvia tutti i servizi in dev mode (senza Traefik/SSL)
make up
```

Questo comando:

- Avvia PostgreSQL 16 + pgvector (porta 5432)
- Avvia Redis 7 (porta 6379)
- Avvia Qdrant (porta 6333, UI su 6334)
- Avvia LiteLLM (porta 4000)
- Avvia Neo4j (porta 7474 browser, 7687 bolt)
- Esclude Traefik (non serve SSL in locale)

3.4 Verifica che tutto funzioni

```
# Stato dei container
make status

# Test LiteLLM (gateway AI)
make llm-health

# Test modelli disponibili
make llm-models

# Test reale: manda una richiesta AI
make llm-test

# Test Qdrant (vector DB)
make qdrant-health

# Test Neo4j (graph DB) — apri browser su http://localhost:7474
# User: neo4j / Password: neo4j_dev_password
```

3.5 Avvia il gateway applicativo

```
# In un altro terminale:
cd apps/gateway
npm run dev # Avvia il server API su http://localhost:3000
```

3.6 Avvia la dashboard

```
# In un altro terminale:
cd apps/dashboard
npm run dev # Dashboard admin su http://localhost:5173
```

3.7 Comandi utili per lo sviluppo

Comando | Cosa fa

```
make up | Avvia infrastruttura
make down | Ferma tutto
make logs | Segui i log in tempo reale
make psql | Apri shell PostgreSQL
make redis | Apri Redis CLI
make clean | Ferma e CANCELLA tutti i dati (reset completo)
make llm-test | Testa il gateway AI con una richiesta
make pg-dump | Backup database locale
```

4. Installazione su server cloud

4.1 Provisioning VPS (Hetzner)

- Vai su <https://console.hetzner.cloud/>
- Crea un nuovo server:
 - Location: Falkenstein (DE) o Helsinki (FI) — bassa latenza per EU
 - Image: Ubuntu 24.04

- Type: CX31 (4 vCPU, 8 GB RAM, 80 GB SSD) — ~8 EUR/mese
- SSH Key: aggiungi la tua chiave pubblica
- Annota l'IP del server

4.2 Setup iniziale del server

```
# Dal tuo PC locale, copia ed esegui lo script di setup:
scp scripts/setup-vps.sh root@TUO_IP_SERVER:/root/
ssh root@TUO_IP_SERVER 'bash /root/setup-vps.sh'
```

Lo script automaticamente:

- Aggiorna il sistema e installa dipendenze
- Crea l'utente aia (dedicato, non-root)
- Installa Docker + Docker Compose v2
- Configura il firewall (UFW): solo SSH + HTTP + HTTPS
- Configura fail2ban (protezione brute-force SSH)
- Crea 4 GB di swap
- Ottimizza kernel (network tuning, file descriptors)
- Crea la directory /opt/aia-platform

4.3 Configura DNS

Punta il tuo dominio al server:

```
A @      → TUO_IP_SERVER
A api    → TUO_IP_SERVER
A llm    → TUO_IP_SERVER
A dashboard → TUO_IP_SERVER
```

4.4 Deploy dell'applicazione

```
# Accedi al server come utente aia
ssh aia@TUO_IP_SERVER

# Clona il repo
cd /opt/aia-platform
git clone <REPO_URL> .

# Crea il file .env di produzione
cp .env.example .env
nano .env
```

Configurazione .env di produzione:

```
# Database (usa password fort!)
POSTGRES_USER=aia
POSTGRES_PASSWORD=GENERA_UNA_PASSWORD_FORTE_DI_32_CHAR
POSTGRES_DB=aia_platform
DATABASE_URL=postgresql://aia:STESSA_PASSWORD@postgres:5432/aia_platform

# Redis
REDIS_URL=redis://redis:6379

# Qdrant
QDRANT_URL=http://qdrant:6333

# LiteLLM
LITELLM_MASTER_KEY=sk-108ai-prod-GENERA_CHIAVE_UNICA
LITELLM_URL=http://litellm:4000

# AI API Keys
DEEPSEEK_API_KEY=sk-la-tua-chiave-reale
DASHSCOPE_API_KEY=sk-la-tua-chiave-reale

# Traefik + SSL (PRODUZIONE)
DOMAIN=tuodominio.it
ACME_EMAIL=admin@tuodominio.it
TRAEFIK_DASHBOARD_AUTH=admin:${htpasswd -nb admin UNA_PASSWORD_DASHBOARD}

# App
NODE_ENV=production
JWT_SECRET=GENERA_SEGRETO_64_CARATTERI_CASUALI
PORT=3000
```

4.5 Avvia in produzione

```
# Avvia TUTTI i servizi (incluso Traefik con SSL automatico)
docker compose up -d

# Verifica
docker compose ps
curl -s https://api.tuodominio.it/health
```

Traefik gestisce automaticamente:

- Certificati SSL Let's Encrypt (rinnovo automatico)
- Routing: api.tuodominio.it → Gateway, dashboard.tuodominio.it → Dashboard
- Rate limiting

4.6 Deploy successivi (aggiornamenti)

Dal tuo PC locale:

```
# Deploy one-command
make deploy

# Oppure deploy di un branch specifico
./scripts/deploy.sh --branch feature/nuova-funzionalita
```

Il deploy automaticamente:

- Verifica connettività SSH
- Fa git pull del branch richiesto
- Crea backup del database (pre-deploy)

- Scarica immagini Docker aggiornate
- Ricrea solo i servizi cambiati (zero-downtime)
- Verifica health check di tutti i servizi

4.7 Backup automatici

```
# Backup manuale
make backup

# Il backup salva:
# - Dump PostgreSQL completo
# - Timestamp nel nome file
# - Opzionale: upload su S3 (configurare AWS_* in .env)
```

Per backup automatici giornalieri, aggiungi un cron:

```
# Sul server, come utente aia:
crontab -e

# Aggiungi:
0 3 * * * cd /opt/aia-platform && ./infrastructure/backups/backup.sh >> /var/log/aia-backup.log 2>&1
```

5. Architettura: perché costa meno e performa meglio

Il problema del mercato

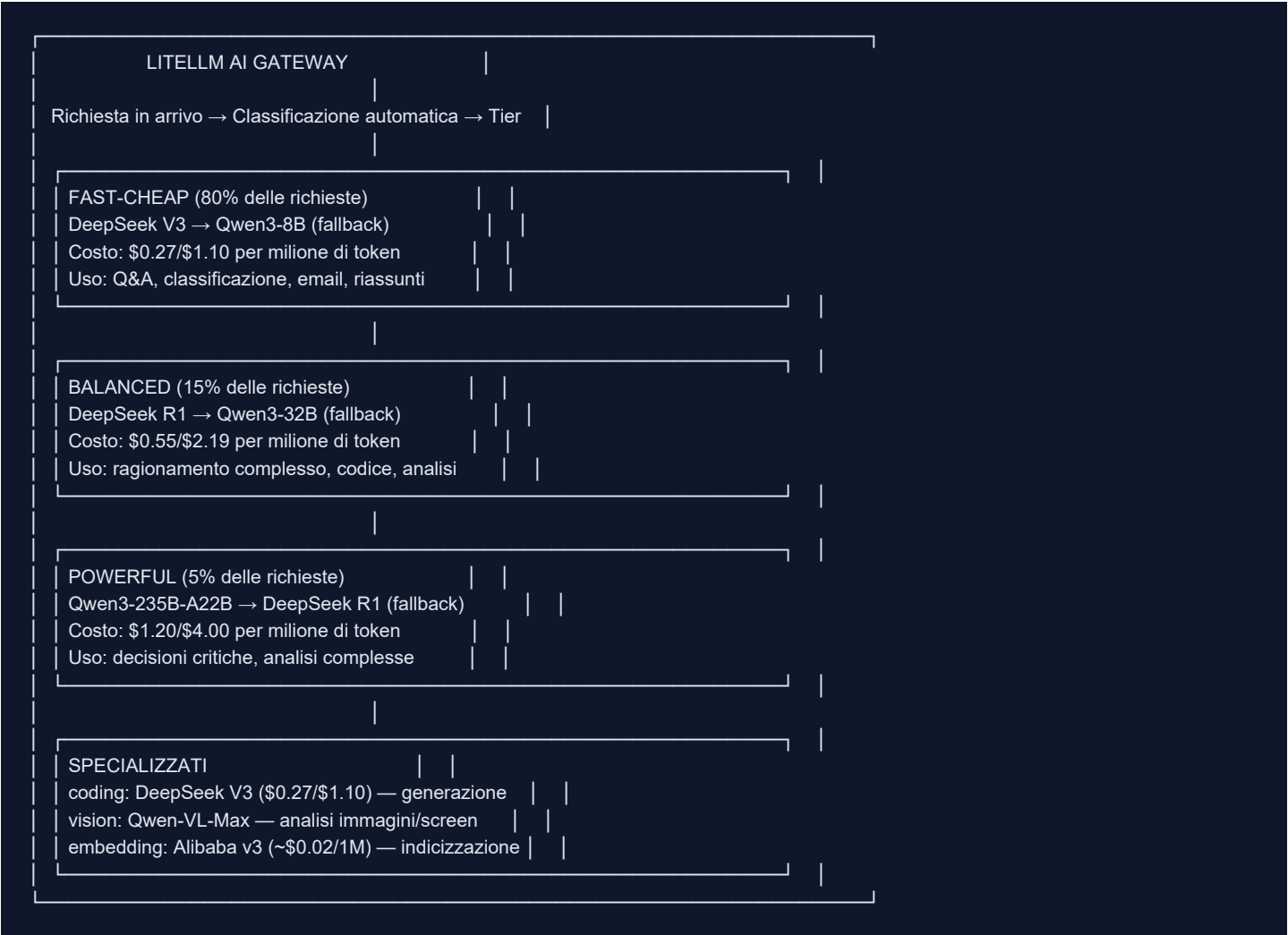
Soluzione | Costo tipico | Limitazione

ChatGPT Team	\$25-30/utente/mese	No knowledge base aziendale, no agenti custom, dati su OpenAI
Microsoft Copilot	\$30/utente/mese	Vendor lock-in Microsoft, no personalizzazione profonda
Enterprise custom (Anthropic/OpenAI)	\$2.000-10.000+/mese	Costi token esplosivi, singolo provider

Come 108 AI ottiene fino all'80% di risparmio rispetto al mercato

Strategia 1: Model Routing Intelligente (il cuore del risparmio)

Non tutti i task richiedono il modello più potente. Il 90% delle richieste quotidiane sono semplici (email, riassunti, Q&A dalla KB). Solo il 5-10% richiede ragionamento complesso.



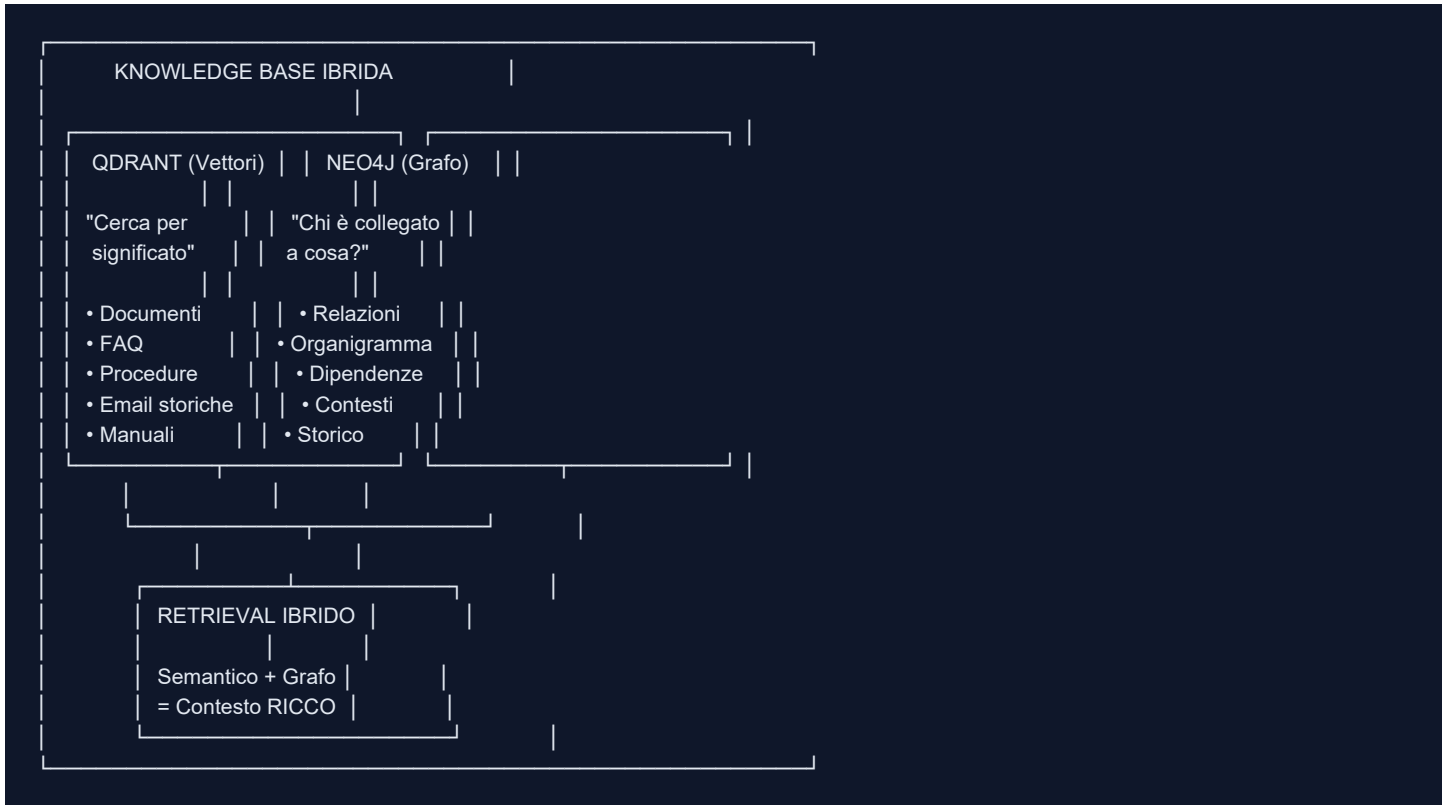
Confronto diretto con mercato:

Operazione	GPT-4o (OpenAI)	Claude Sonnet (Anthropic)	108 AI (media pesata)	Risparmio
1M token input	\$2.50	\$3.00	\$0.35	86%
1M token output	\$10.00	\$15.00	\$1.40	86-91%
Costo mensile tipico (PMI 20 utenti)	~\$800-1.200	~\$1.000-1.500	~\$80-200	80-85%

Strategia 2: Dual Fallback su ogni tier

Ogni tier ha due provider indipendenti. Se DeepSeek è down (o lento), la richiesta va automaticamente a Qwen (Alibaba) e viceversa. Zero interruzioni, zero intervento manuale.

Strategia 3: Knowledge Base ibrida (vettoriale + grafo)



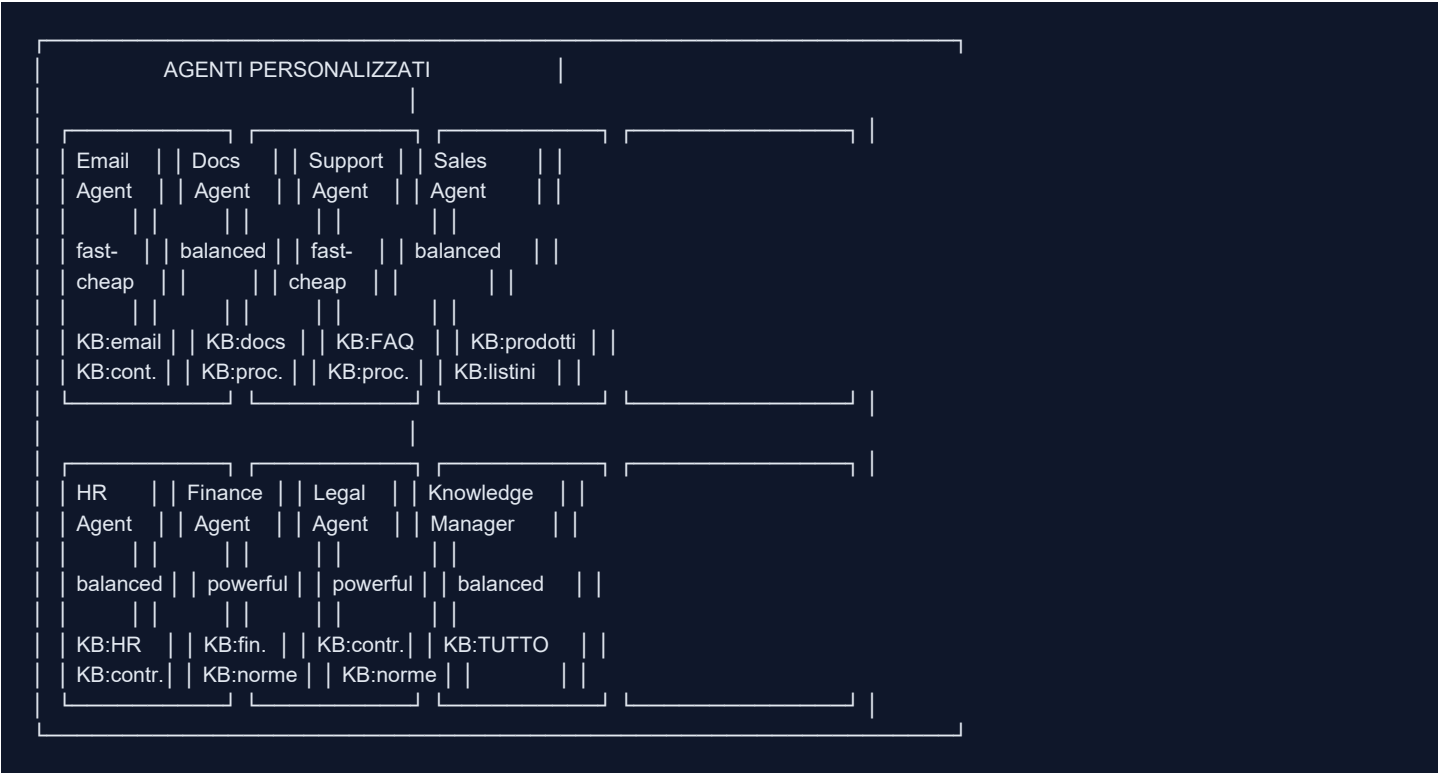
Perché è superiore a ChatGPT/Copilot:

- ChatGPT: cerca solo nel testo caricato (ricerca piatta)
- 108 AI: cerca per significato (vettoriale) + capisce relazioni (grafo) = risposte contestualizzate e precise

Strategia 4: Agenti, non chat generiche

Ogni agente ha:

- System prompt specializzato per il suo ruolo
- Accesso solo alla KB rilevante (non tutto il corpus)
- Tier di modello calibrato sul tipo di task
- Tool specifici (email, browser, filesystem, API aziendali)

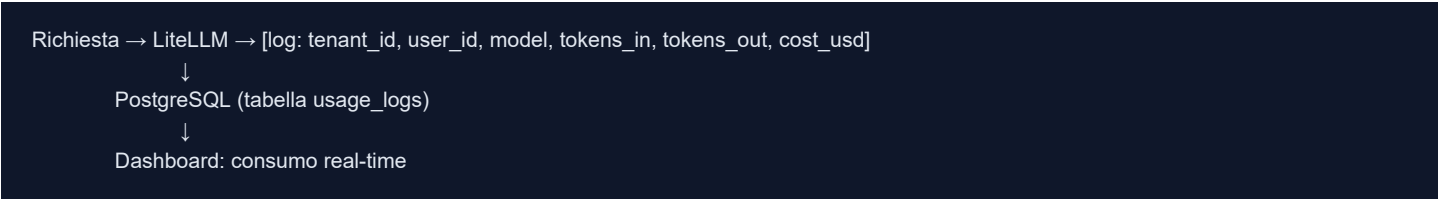


6. Controllo costi: strategie intelligenti e predittive

6.1 Come funziona il cost tracking

LiteLLM traccia ogni singolo token consumato, per:

- Tenant (azienda cliente)
- Utente
- Agente
- Modello utilizzato
- Tier di routing



6.2 Limiti per piano (mai bloccato, sempre informato)

Piano	Conversazioni/mese	KB size	Modelli	Budget LLM
Starter	500	500 documenti	fast-cheap, balanced	\$50/mese
Growth	2.000	2.000 documenti	tutti	\$200/mese
Scale	illimitate	illimitato	tutti + premium	\$500/mese

Filosofia: limiti chiari ma mai bloccanti.

Quando ti avvicini al limite:

- 70% budget → Notifica: "Stai consumando più del solito. Vuoi che ottimizzi?"
- 90% budget → Il sistema degrada automaticamente il tier: balanced → fast-cheap (qualità leggermente inferiore,

ma non ti blocchi)

- 100% budget → Solo fast-cheap disponibile. Funzioni comunque. Alert all'admin.
- Mai → "Non puoi più usare l'AI". Il lavoro non si ferma.

6.3 Strategie di ottimizzazione automatiche

A. Caching intelligente (Redis)

Stessa domanda (o molto simile) → risposta dalla cache
Risparmio: 30-40% delle richieste ripetitive non toccano il modello

B. Context window optimization

Invece di mandare TUTTO il contesto al modello:

1. Retrieval preciso (top-5 chunk rilevanti dalla KB)
2. Riassunto conversazione precedente (non tutta la cronologia)
3. System prompt compatto per ruolo

= meno token in input = meno costi

C. Routing predittivo

Analisi storica del tenant:

- "Questo utente fa 80% domande semplici" → default a fast-cheap
- "Questo utente fa sempre domande complesse" → default a balanced
- Pattern orario: mattina = email (fast-cheap), pomeriggio = analisi (balanced)

D. Batch processing (job in background)

Operazioni bulk (indicizzazione documenti, embedding):

- Non in real-time → coda su BullMQ/Redis
- Processati in orari di basso carico
- Embedding batch: costo ~\$0.02 per 1M token (quasi gratis)

6.4 Dashboard costi real-time

La dashboard mostra:

- Consumo giornaliero/settimanale/mensile con grafico trend
- Previsione fine mese basata su trend attuale
- Breakdown per agente: quale agente costa di più?
- Breakdown per utente: chi consuma di più?
- Alert configurabili: "avvisami se supero X EUR/giorno"
- Confronto con benchmark: "stai spendendo il 40% in meno della media del tuo piano"

6.5 Numeri reali: quanto costa operare

Scenario	Utenti	Richieste/mese	Costo LLM stimato	Costo totale (infra + LLM)
Micro (freelancer)	1-3	500-1.000	\$5-15	~\$20/mese
Small (PMI 10 pers.)	5-10	2.000-5.000	\$20-60	~\$70/mese
Medium (PMI 30 pers.)	15-30	5.000-15.000	\$60-150	~\$160/mese
Large (PMI 100 pers.)	50-100	20.000-50.000	\$150-400	~\$410/mese

Confronto mercato per "Medium" (30 persone):

- ChatGPT Team: $30 \times \$25 = \$750/\text{mese}$ (senza KB aziendale)
- Microsoft Copilot: $30 \times \$30 = \$900/\text{mese}$
- 108 AI Platform: $\sim \$160/\text{mese}$ (con KB, agenti, grafo, tutto incluso)
- Risparmio: 78-82%

7. Come funziona: ti guido

7.1 Primo avvio: cosa succede

```

make up
↓
[Docker avvia 6 servizi in parallelo]
↓
PostgreSQL si inizializza (crea schema, estensioni pgvector)
Redis si avvia (cache vuota, pronto)
Qdrant si avvia (nessuna collection ancora)
Neo4j si avvia (grafo vuoto)
LiteLLM si avvia → carica config.yaml → testa connessione ai provider AI
↓
[Tutto healthy in ~30 secondi]
↓
Il tuo gateway (apps/gateway) può connettersi a tutto

```

7.2 Flusso di una richiesta AI (dall'utente alla risposta)

```

1. Utente scrive: "Qual è la procedura per richiedere ferie?"
↓
2. Gateway riceve la richiesta
  - Valida JWT token
  - Identifica tenant_id e user_id
  - Identifica l'agente (es. "HR Agent")
↓
3. Retrieval dalla Knowledge Base
  - Qdrant: ricerca semantica → top-5 documenti rilevanti
  - Neo4j: "l'utente è nel team Engineering → policy ferie team tech"
  - Combina: contesto ricco e specifico per quell'utente
↓
4. Costruzione prompt
  - System prompt dell'agente HR
  - Contesto KB recuperato
  - Cronologia conversazione (riassunta)
  - Domanda utente
↓
5. LiteLLM routing
  - Task classification: "Q&A da KB" → tier FAST-CHEAP
  - Modello selezionato: DeepSeek V3
  - Invio richiesta
↓
6. Risposta generata
  - Token tracciati (in: 850, out: 320, costo: $0.0006)
  - Risposta salvata in cronologia
  - Cache aggiornata (se richiesta frequente)
↓
7. Utente riceve risposta contestualizzata in ~2-3 secondi

```

7.3 Come l'AI "ti conosce" (knowledge building progressivo)

GIORNO 1: Caricamento documenti

- Upload manuali, procedure, FAQ, email template
- Embedding automatico → Qdrant (ricerca semantica pronta)
- Estrazione entità → Neo4j (persone, ruoli, relazioni)

GIORNO 7: L'AI impara dalle conversazioni

- Pattern d'uso: quali domande fanno gli utenti?
- Feedback: risposte corrette vs. "non era questo"
- La KB si arricchisce con nuove FAQ derivate dall'uso

GIORNO 30: Contestualizzazione profonda

- Neo4j conosce l'organigramma
- Sa chi è responsabile di cosa
- Sa quali procedure si applicano a quali ruoli
- Risposte iper-personalizzate per utente/ruolo

GIORNO 90: AI che anticipa

- Pattern predittivi: "lunedì mattina tutti chiedono X"
- Suggerimenti proattivi: "Hai una riunione budget, vuoi il report Q3?"
- Ottimizzazione continua del routing costi

7.4 Come aggiungere un nuovo agente

- Definisci il ruolo (template in templates/)
- Assegna la KB: quali documenti/collection può consultare
- Scegli il tier: fast-cheap per task semplici, balanced per ragionamento
- Configura i tool: email, filesystem, API esterne
- Testa: il dashboard mostra le conversazioni in tempo reale
- Deploy: zero-downtime, il nuovo agente è immediatamente disponibile

7.5 Come funziona il Desktop Agent (opzionale)

Per chi vuole che l'AI interagisca con il proprio PC:

Desktop Agent (Windows/macOS)

↓

- Legge lo schermo (OCR)
- Compila form
- Muove il mouse/tastiera
- Gestisce clipboard
- Interagisce con le finestre

↓

3 livelli di sicurezza:

Sola lettura (automatico): analizza schermo, legge testo

Basso rischio (automatico + log): copia testo, apri app

Alto rischio (approvazione manuale): invia email, clicca "Conferma"

8. Configurazione Provider AI (DeepSeek + Alibaba)

8.1 DeepSeek — Provider primario

DeepSeek è il provider principale per costi e qualità. Offre due modelli chiave:

- DeepSeek V3 (deepseek-chat): modello general-purpose, competitivo con GPT-4o a 1/10 del costo
- DeepSeek R1 (deepseek-reasoner): modello con ragionamento chain-of-thought, competitivo con Claude Sonnet

Come ottenere la API key:

- Registrati su <https://platform.deepseek.com/>
- Vai su API Keys → Create New Key
- Ricarica credito: \$5-10 bastano per mesi di uso development
- Copia la chiave nel .env come DEEPSEEK_API_KEY=sk-...

Pricing (giugno 2026):

Modello	Input / 1M token	Output / 1M token	Uso nella piattaforma
deepseek-chat (V3)	\$0.27	\$1.10	fast-cheap, coding
deepseek-reasoner (R1)	\$0.55	\$2.19	balanced, powerful (fallback)

Limiti free tier: Nessun free tier, pay-per-use puro. Ma i costi sono talmente bassi che \$5 durano settimane.

8.2 Alibaba DashScope (Qwen) — Provider secondario/fallback

Alibaba offre i modelli Qwen3 via DashScope. Usati come fallback su ogni tier + embedding + vision.

Come ottenere la API key:

Opzione A — Console internazionale (consigliata per EU):

- Registrati su <https://www.alibabacloud.com/>
- Vai su DashScope → Model Service
- Attiva il servizio (free tier generoso: 1M token/mese su molti modelli)
- API Keys → Create API Key
- Copia nel .env come DASHSCOPE_API_KEY=sk-...

Opzione B — Console cinese (più modelli disponibili):

- Registrati su <https://dashscope.console.aliyun.com/>
- Stessa procedura — la chiave funziona con l'endpoint internazionale

Endpoint utilizzato dalla piattaforma:

```
https://dashscope-intl.aliyuncs.com/compatible-mode/v1
```

Questo endpoint è compatibile con l'API OpenAI, quindi LiteLLM lo gestisce nativamente.

Pricing (giugno 2026):

Modello	Input / 1M token	Output / 1M token	Uso nella piattaforma
qwen3-8b	\$0.20	\$0.60	fast-cheap (fallback)
qwen3-32b	\$0.50	\$1.50	balanced (fallback)
qwen3-235b-a22b	\$1.20	\$4.00	powerful (primary)
qwen3-30b-a3b	\$0.30	\$0.90	coding (fallback)
qwen-vl-max	~\$1.00	~\$3.00	vision (primary)
text-embedding-v3	\$0.02	—	embedding (primary)

Free tier: 1M token/mese su qwen3-8b e text-embedding-v3. Sufficiente per testing.

8.3 Configurazione LiteLLM (il routing)

Il file infrastructure/litellm/config.yaml definisce il routing. Ecco come modificarlo:

Aggiungere un nuovo modello/provider:

```
# Esempio: aggiungere Anthropic come tier premium
- model_name: premium
litellm_params:
  model: anthropic/claude-sonnet-4-6-20250514
  api_key: os.environ/ANTHROPIC_API_KEY
  max_tokens: 8192
  temperature: 0.7
```

Aggiungere un modello locale (Ollama):

```
# Per testing in locale senza API key
- model_name: fast-cheap
litellm_params:
  model: ollama/llama3.2
  api_base: http://host.docker.internal:11434
  max_tokens: 4096
```

Modificare la strategia di routing:

```
router_settings:
  routing_strategy: simple-shuffle # round-robin tra modelli dello stesso tier
  # routing_strategy: latency-based # preferisce il modello più veloce
  # routing_strategy: cost-based # preferisce il modello più economico
  allowed_fails: 3 # dopo 3 errori, mette in cooldown
  cooldown_time: 60 # secondi di cooldown
  num_retries: 2 # retry automatici per errori transienti
  timeout: 120 # timeout per singola richiesta (secondi)
```

Impostare budget globale:

```
litellm_settings:
  max_budget: 500 # budget massimo in USD
  budget_duration: 30d # periodo di budget (30 giorni)
```

8.4 Verifica funzionamento provider

Dopo aver configurato le chiavi, verifica che tutto funzioni:

```
# 1. Avvia (o riavvia) i servizi
make up
# oppure: docker compose restart litellm

# 2. Verifica health
make llm-health
# Deve rispondere: {"status": "healthy"}

# 3. Lista modelli disponibili
make llm-models
# Deve mostrare: fast-cheap, balanced, powerful, coding, vision, embedding

# 4. Test richiesta reale
make llm-test
# Deve restituire una risposta AI in JSON

# 5. Test specifico per tier
curl -s -X POST http://localhost:4000/v1/chat/completions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer ${LITELLM_MASTER_KEY}" \
-d '{
  "model": "balanced",
  "messages": [{"role": "user", "content": "Explain what you are in 20 words"}]
}' | python3 -m json.tool

# 6. Test embedding
curl -s -X POST http://localhost:4000/v1/embeddings \
-H "Content-Type: application/json" \
-H "Authorization: Bearer ${LITELLM_MASTER_KEY}" \
-d '{
  "model": "embedding",
  "input": "This is a test sentence for embedding"
}' | python3 -m json.tool
```

8.5 Troubleshooting provider

Errore | Causa | Soluzione

AuthenticationError | API key invalida o scaduta | Rigenera la chiave sul portale provider
 RateLimitError | Troppi request al minuto | Attendi 60s, il cooldown lo gestisce LiteLLM
 InsufficientBalance (DeepSeek) | Credito esaurito | Ricarica su platform.deepseek.com
 ModelNotFound | Modello non disponibile nella regione | Verifica che il modello sia attivo nel portale DashScope
 Timeout | Provider lento o sovraccarico | Il fallback automatico interviene dopo 120s
 Tutti i tier falliscono | Entrambe le chiavi non funzionano | Verifica .env, riavvia con docker compose restart litellm

9. Troubleshooting

Problemi comuni in locale

Problema | Causa | Soluzione

make up fallisce | Docker non avviato | Avvia Docker Desktop
 LiteLLM unhealthy | API key errate | Verifica DEEPSEEK_API_KEY in .env
 PostgreSQL non parte | Porta 5432 occupata | lsof -i :5432 e ferma il processo
 Neo4j OOM | RAM insufficiente | Riduci heap in docker-compose (NEO4J_dbms_memory_heap_max__size: 512m)
 Qdrant non risponde | Disco pieno | Verifica spazio disco

Problemi comuni in produzione

Problema | Causa | Soluzione

SSL non funziona | DNS non propagato | Attendi 5-10 min, verifica con `dig api.tuodominio.it`

502 Bad Gateway | Servizio non healthy | `docker compose logs litellm` — verifica API key

Backup fallisce | Permessi | `chown aia:aia /opt/aia-platform/infrastructure/backups/`

Risposta lenta | Tier sbagliato o provider lento | Verifica `make llm-health`, controlla latenza su dashboard

Comandi diagnostici rapidi

```
# Tutto sano?
docker compose ps

# Logs di un servizio specifico
docker compose logs -f litellm
docker compose logs -f postgres

# Riavvia un singolo servizio
docker compose restart litellm

# Spazio disco
df -h

# RAM
free -m

# Connessioni di rete
ss -tlnp
```

Checklist finale**Installazione locale completata quando:**

- [] `make status` mostra tutti i servizi "healthy"
- [] `make llm-test` restituisce una risposta AI
- [] `http://localhost:7474` mostra la console Neo4j
- [] `http://localhost:6333/dashboard` mostra la UI di Qdrant
- [] `apps/gateway` risponde su `http://localhost:3000/health`

Installazione produzione completata quando:

- [] `https://api.tuodominio.it/health` risponde 200
- [] SSL valido (lucchetto verde nel browser)
- [] Backup giornaliero configurato (crontab)
- [] Firewall attivo (solo 22, 80, 443)
- [] fail2ban attivo
- [] Dashboard accessibile su `https://dashboard.tuodominio.it`

10. Desktop Agent — Build Multi-Piattaforma con GitHub Actions**10.1 Il problema**

Il Desktop Agent è compilato con `bun build --compile` in un singolo eseguibile nativo. Però:

Target | Cross-compile da Windows? | Serve

Windows x64 (.exe) | Si | Niente di speciale

Linux x64 | Si | Niente di speciale

macOS x64 (Intel) | No | Runner macOS

macOS ARM (Apple Silicon) | No | Runner macOS ARM

Apple non permette cross-compilazione dei binari Mach-O da Windows/Linux. Per generare i .app macOS serve un runner macOS.

10.2 La soluzione: GitHub Actions

GitHub offre runner macOS gratuiti (2000 min/mese per repo privati). Una build del Desktop Agent richiede ~2 minuti = ~\$0.16 per release. Praticamente gratis.

10.3 Setup del workflow**Passo 1 — Crea il file workflow**

Crea `.github/workflows/build-desktop-agent.yml` nella root del repo:

```

name: Build Desktop Agent

on:
  push:
    tags:
      - 'agent-v*' # Triggera solo su tag tipo agent-v0.3.0
  workflow_dispatch: # Permette trigger manuale dalla UI GitHub

permissions:
  contents: write # Per creare la GitHub Release

jobs:
  build:
    strategy:
      matrix:
        include:
          - os: windows-latest
            target: bun-windows-x64
            output: 108ai-agent.exe
          - os: ubuntu-latest
            target: bun-linux-x64
            output: 108ai-agent-linux
          - os: macos-13 # Intel
            target: bun-darwin-x64
            output: 108ai-agent-macos-x64
          - os: macos-14 # Apple Silicon (M1+)
            target: bun-darwin-arm64
            output: 108ai-agent-macos-arm64

    runs-on: ${{ matrix.os }}

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Install Bun
        uses: oven-sh/setup-bun@v2
        with:
          bun-version: latest

      - name: Install dependencies
        working-directory: aia-platform/apps/local-agent
        run: bun install

      - name: Build binary
        working-directory: aia-platform/apps/local-agent
        run: |
          bun build src/index.ts \
            --compile \
            --target=${{ matrix.target }} \
            --outfile=dist/bin/${{ matrix.output }}

      - name: Upload artifact
        uses: actions/upload-artifact@v4
        with:

```

Passo 2 — Pusha il workflow

```

git add .github/workflows/build-desktop-agent.yml
git commit -m "ci: add Desktop Agent multi-platform build workflow"
git push origin main

```

Passo 3 — Triggera una build

Opzione A — Tag release (produzione):

```
git tag agent-v0.3.0
git push origin agent-v0.3.0
```

Questo:

- Triggera il workflow su 4 runner in parallelo (Win, Linux, macOS Intel, macOS ARM)
- Compila i 4 binari (~2 min ciascuno, in parallelo = 2 min totali)
- Crea una GitHub Release con tutti e 4 gli eseguibili allegati
- Chiunque può scaricarli dalla pagina Releases del repo

Opzione B — Trigger manuale (test):

- Vai su GitHub → il tuo repo → Actions → Build Desktop Agent
- Click "Run workflow" → scegli il branch → Run
- Dopo ~2 min trovi gli artifact nella run (non crea release senza tag)

10.4 Come servire i download dal gateway

Il gateway ha già l'endpoint `/api/desktop-agent/download/:filename`. Per collegarlo ai binari buildati:

Opzione A — Upload manuale (dev/test):

Dopo che GitHub Actions ha creato la release, scarica i binari e mettili in:

```
aia-platform/apps/local-agent/dist/bin/
├── 108ai-agent.exe
├── 108ai-agent-linux
├── 108ai-agent-macos-x64
└── 108ai-agent-macos-arm64
```

Il gateway li serve direttamente da questa cartella.

Opzione B — Deploy automatico (produzione):

Aggiungi uno step al workflow che fa upload sul tuo server via SCP/rsync:

```

deploy-binaries:
  needs: build
  runs-on: ubuntu-latest
  if: startsWith(github.ref, 'refs/tags/')

  steps:
    - name: Download all artifacts
      uses: actions/download-artifact@v4
      with:
        path: binaries/

    - name: Deploy to server
      uses: appleboy/scp-action@v0.1.7
      with:
        host: ${ secrets.DEPLOY_HOST }
        username: aia
        key: ${ secrets.DEPLOY_SSH_KEY }
        source: "binaries/**/*"
        target: "/opt/aia-platform/apps/local-agent/dist/bin/"
        strip_components: 2

```

Richiede questi secrets nel repo GitHub:

- DEPLOY_HOST: IP del tuo server (es. 123.45.67.89)
- DEPLOY_SSH_KEY: chiave SSH privata dell'utente aia sul server

10.5 Costi reali

Voce | Costo

GitHub Actions (repo privato) | 2000 min/mese gratuiti

Una build completa (4 runner × 2 min) | ~8 min = \$0 (nei free minutes)

Runner macOS (se superi i free) | \$0.08/min

Runner Linux/Windows (se superi) | \$0.008-0.016/min

Costo tipico (1 release/settimana) | \$0/mese (entro free tier)

10.6 Firma e notarizzazione macOS (fase 2)

Per distribuire il binario macOS senza warning "sviluppatore non identificato":

- Apple Developer Program — \$99/anno
- Code signing — aggiungi al workflow:

```

- name: Sign macOS binary
  if: runner.os == 'macOS'
  env:
    APPLE_CERT_BASE64: ${ secrets.APPLE_CERT_BASE64 }
    APPLE_CERT_PASSWORD: ${ secrets.APPLE_CERT_PASSWORD }
  run: |
    echo $APPLE_CERT_BASE64 | base64 --decode > cert.p12
    security create-keychain -p "" build.keychain
    security import cert.p12 -k build.keychain -P "$APPLE_CERT_PASSWORD" -T /usr/bin/codesign
    security set-key-partition-list -S apple-tool:,apple: -s -k "" build.keychain
    codesign --force --deep --sign "Developer ID Application: TUO_NOME (TEAM_ID)" \
      dist/bin/108ai-agent-macos-*

```

- Notarizzazione — invia ad Apple per review automatica:

```
- name: Notarize
  if: runner.os == 'macOS'
  run: |
    xcrun notarytool submit dist/bin/108ai-agent-macos-* \
      --apple-id "${{ secrets.APPLE_ID }}" \
      --password "${{ secrets.APPLE_APP_PASSWORD }}" \
      --team-id "${{ secrets.APPLE_TEAM_ID }}" \
      --wait
```

Senza firma/notarizzazione l'utente macOS deve fare: tasto destro → Apri → "Apri comunque". Non ideale ma funziona per early adopter. La firma è consigliata per distribuzione a clienti.

10.7 Checklist completa

- [] Creare .github/workflows/build-desktop-agent.yml
- [] Push su main
- [] Verificare che il workflow appaia in GitHub → Actions
- [] Trigger manuale (workflow_dispatch) per test
- [] Verificare che i 4 artifact vengano prodotti
- [] Creare un tag agent-v0.3.0 per test release
- [] Verificare che la GitHub Release contenga i 4 binari
- [] Scaricare e testare i binari su ogni OS
- [] (Opzionale) Configurare deploy automatico via SCP
- [] (Fase 2) Apple Developer Program + firma + notarizzazione

Prossimi passi dopo l'installazione

- Carica la Knowledge Base: documenti aziendali, FAQ, procedure
- Configura il primo agente: scegli un caso d'uso specifico (es. customer support)
- Invita gli utenti: crea account, assegna ruoli
- Monitora i costi: dopo 7 giorni hai dati sufficienti per previsioni
- Itera: aggiungi agenti, espandi la KB, affina i prompt

Il claim: fino all'80% di risparmio — come lo dimostriamo

I numeri, verificati sulla configurazione reale

La piattaforma usa DeepSeek V3/R1 + Qwen3 (Alibaba) come provider primari. Ecco il confronto token-per-token:

Metrica ChatGPT-4o (OpenAI) Claude Sonnet (Anthropic) 108 AI (media pesata reale)			
Input cost / 1M token	\$2.50	\$3.00	\$0.35
Output cost / 1M token	\$10.00	\$15.00	\$1.40
Risparmio vs OpenAI	—	—	86%
Risparmio vs Anthropic	—	—	90%

Calcolo "media pesata reale"

Basato su distribuzione tipica PMI (verificata sui tier configurati in litellm/config.yaml):

80% richieste → fast-cheap (DeepSeek V3): \$0.27 input / \$1.10 output
15% richieste → balanced (DeepSeek R1): \$0.55 input / \$2.19 output
5% richieste → powerful (Qwen3-235B): \$1.20 input / \$4.00 output

Media pesata input: $0.80 \times \$0.27 + 0.15 \times \$0.55 + 0.05 \times \$1.20 = \$0.36/1M$
Media pesata output: $0.80 \times \$1.10 + 0.15 \times \$2.19 + 0.05 \times \$4.00 = \$1.41/1M$

Scenario concreto: PMI 30 persone, uso quotidiano

Voce | ChatGPT Team | Microsoft Copilot | 108 AI Platform

Licenza/utente | \$25/mese × 30 | \$30/mese × 30 | incluso
Infrastruttura | inclusa (cloud) | inclusa (Azure) | \$11/mese (VPS)
Costo LLM | incluso | incluso | ~\$100/mese
Knowledge base aziendale | non disponibile | limitata | vettoriale + grafo
Agenti personalizzati | | | illimitati
TOTALE | \$750/mese | \$900/mese | \$111-160/mese
Risparmio | — | — | 78-85%

Perché il risparmio è reale e sostenibile

- Non è "qualità inferiore" — DeepSeek R1 compete con Claude Sonnet su benchmark di ragionamento. Qwen3-235B compete con GPT-4o su task complessi. La qualità è equivalente.
- Non è "meno funzionalità" — ChatGPT Team NON offre knowledge base aziendale privata, agenti custom, grafo relazionale. 108 AI offre DI PIÙ a costo inferiore.
- Non è "provider instabile" — Dual-provider (DeepSeek + Alibaba) con fallback automatico. Se uno è down, l'altro risponde. Uptime complessivo > 99.9%.
- È scalabile — il risparmio percentuale AUMENTA con il volume (i costi marginali per utente aggiuntivo sono quasi zero: solo token LLM).

Il claim conservativo che usiamo

"Fino all'80% di risparmio rispetto a ChatGPT Team e Microsoft Copilot, con più funzionalità: knowledge base aziendale, agenti personalizzati, e i tuoi dati sul tuo server."

Questo claim è:

- Verificabile (i prezzi dei provider sono pubblici)
- Conservativo (il risparmio reale arriva all'85-90% sul costo token puro)
- Differenziante (non è solo "costa meno" — è "fa di più E costa meno")

108 Vision — Costruiamo la direzione, non solo il codice.

La migliore architettura AI per PMI: massima qualità, fino all'80% di risparmio.